从Harness到后训练：以GUI测试执行Agent为例的领域Agent优化实践

演讲嘉宾：詹伶俐

领域 Agent 优化：用 Harness，还是后训练?

这是一个开放性、且在持续技术演进的问题——不同时间点、场景与需求，答案并不相同。本分享不聚焦如何做 GUI 测试执行 Agent，而是借这一案例，从实践角度提供领域 Agent 优化的思考与经验参考。

为什么以 GUI 测试执行 Agent 为例

01

GUI Agent：典型倚重 Harness 的任务

操作链路长、界面反馈即时，能力上限很大程度由提示、工具与流程编排等 Harness 决定。

02

测试任务：天然具备丰富场景数据

相比单纯 GUI Agent，测试案例天然沉淀了大量真实、可复用的场景（测试案例）数据。

本次分享核心议题主线

主线一

通过 Harness Engineering 优化领域 Agent

从提示、工具、记忆与流程编排入手，系统性打磨领域 Agent 的工程外壳。

主线二

领域 Agent 如何做模型后训练

破局「你先给我十万条优质数据」，探索低成本、可落地的数据与训练路径。

目录

CONTENT

1 如何通过 Harness Engineering 优化领域 Agent

2 领域Agent如何做模型后训练--破局“你先给我十万条优质数据”

PART 1

如何通过 Harness Engineering 优化领域 Agent

01 Agent 的本质

为什么可以通过 Harness 优化 Agent？

02 Harness 干预的 10 个节点

哪些可以调整？各自有什么特性？

03 GUI 测试执行 Agent 的 Harness Engineering 工作复盘

怎么调整（一）：从实践中拆解优化过程

怎么调整（二）：上下文工程

01 Agent 的本质

为什么可以通过 Harness 优化 Agent？

领域 **Agent** 优化

模型不是 **Agent**，闭环才是 **Agent**

Harness 是Agent的脚手架，是把输入、动作、状态、验证和恢复变成可控的工程接口。

Agent = 模型能力 + 上下文 + 工具 + 状态 + 反馈 + 约束

Harness = Loop Maintenance：循环调用模型，直到模型不再调用工具并返回文本。



模型每一轮只有两个决定：调用工具 / 不调用工具

先定位问题，再决定路线

三类能力，三种优化位置

方案	主要解决什么	GUI Agent 测试执行中的典型问题
RAG / 本体	知道什么	业务规则、页面语义、测试数据、领域知识如何被检索和注入。
Harness	怎么被约束地执行	任务编排、工具 schema、状态同步、动作校验、异常恢复、输出验证。
后训练	模型内化什么行为模式	跨页面复用的任务分解、工具策略、状态判断和恢复能力。

如果问题来自编排、约束、状态或校验，优先在 **Harness** 侧解决。

loop 相当于 **rollout**：真实环境反馈驱动策略调整，本身就是一种 **Agent** 式强化学习。

知识库再完备，仍然是静态的；**Agent** 的稳定性来自可观察、可验证、可恢复的交互闭环。

02 Harness 干预的 10 个节点

哪些可以调整？各自有什么特性？

Harness 干预的 10 个节点

Trace 是一次完整任务；Turn 是其中一次模型调用。Harness 贯穿两者之间的每个工程钩子。



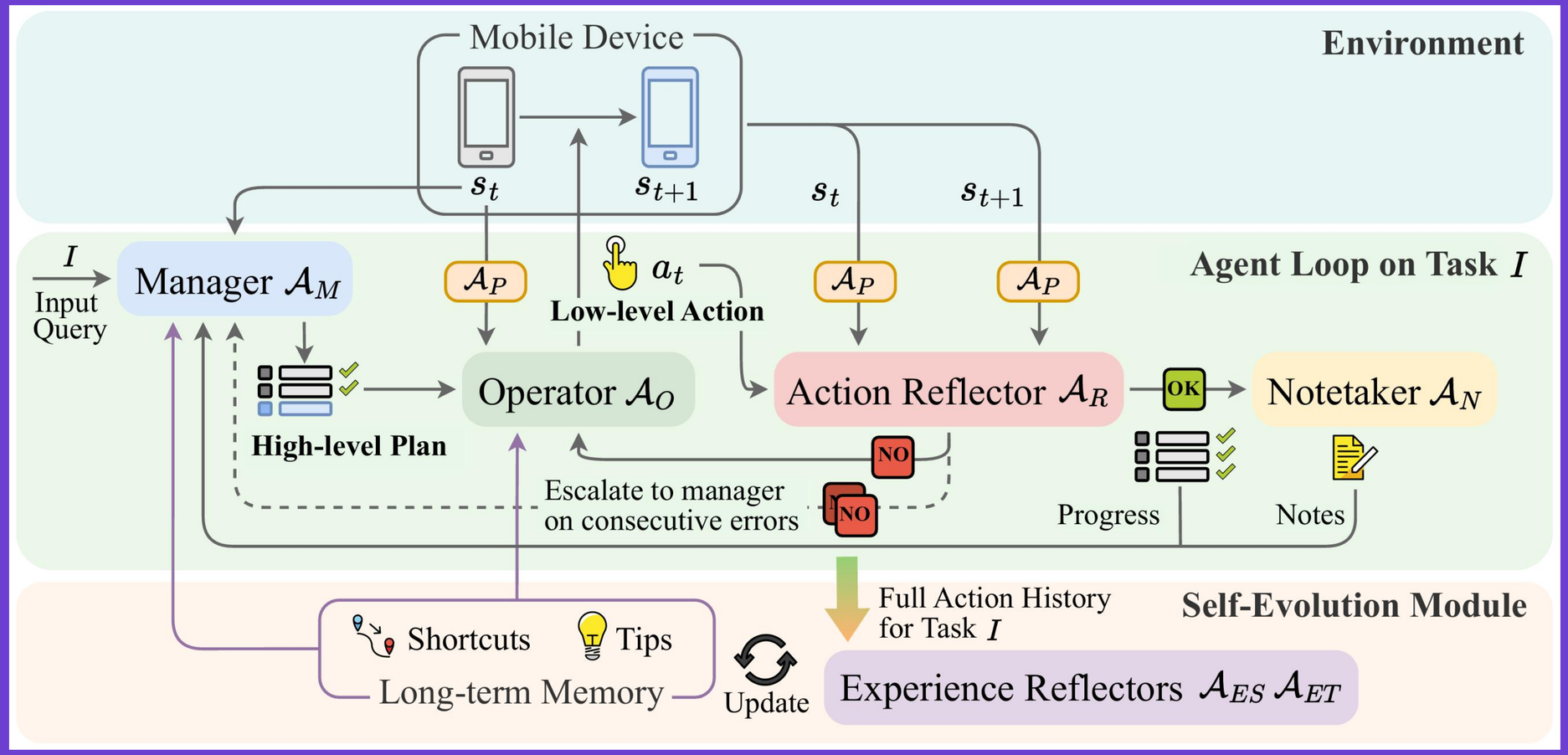
Harness 不是一个单点模块，而是贯穿 Trace 和每个 Turn 的工程控制层。

03 GUI 测试执行 Agent 的 Harness Engineering 工作复盘

怎么调整（一）：从实践中拆解优化过程

怎么调整（二）：上下文工程

GUI AGENT 架构


Environment
Mobile Device
s_t
s_{t+1}
s_t
s_{t+1}
Agent Loop on Task I
I
Input Query
Manager $\mathcal{A}_M$
$\mathcal{A}_P$
a_t
Low-level Action
$\mathcal{A}_P$
$\mathcal{A}_P$
Operator $\mathcal{A}_O$
Action Reflector $\mathcal{A}_R$
OK
Notetaker $\mathcal{A}_N$
High-level Plan
NO
Escalate to manager on consecutive errors
NO
Progress
Notes
Full Action History for Task I
Self-Evolution Module
Shortcuts
Tips
Long-term Memory
Update
Experience Reflectors $\mathcal{A}_{ES}$ $\mathcal{A}_{ET}$

在原有 harness 上优化适配

1 日志

核心要点

- 按测试场景重新设计元数据和索引
- 以便于观测为原则可冗余记录
- 配套检测和格式转换脚本

2 提示词

核心要点

- 模仿大模型官方示例表述风格
- 考虑 KV 缓存，分清共性/个性位置
- 整体抽象简练，易错点反复强调

3 工具与调用

核心要点

- 根据场景构建自己的执行工具
- 基于场景语义封装组合工具代替推理
- 不同 action 之间尽量保持正交
- 对调用结果做后处理，保成功率可监控

4 上下文构建

核心要点

- 适配自己的执行工具接口
- 调控历史帧/轨迹/goal/subgoal 投放
- 保留明确记忆，删除不确定记忆
- 可做小型检索知识库动态加载

5 多智能体设定

核心要点

- 控制数量，能一个做别设计成两个
- 角色/工具差异大又聚焦的可拆分
- 串行依赖低、可异步的可拆开
- 降低附属体置信度，冲突以执行体为主

6 loop 控制与人在回路

核心要点

- 设置异常触发机制
- 人机协同，一人维护多台设备
- 以效率为优先设置重试机制
- 结果校验与规则兜底

三条实践经验

1 建立固定测试集

固定评测样本与评价标准，效果可衡量可比较

2 完善日志系统

记录执行轨迹与工具调用，定位失败节点

3 控制变量

每次聚焦一个变量，对照评测验证后保留或回退

上下文工程
决定 Agent 的上限

Agent 的效果，取决于模型能力，也取决于每一轮提交给模型的上下文。

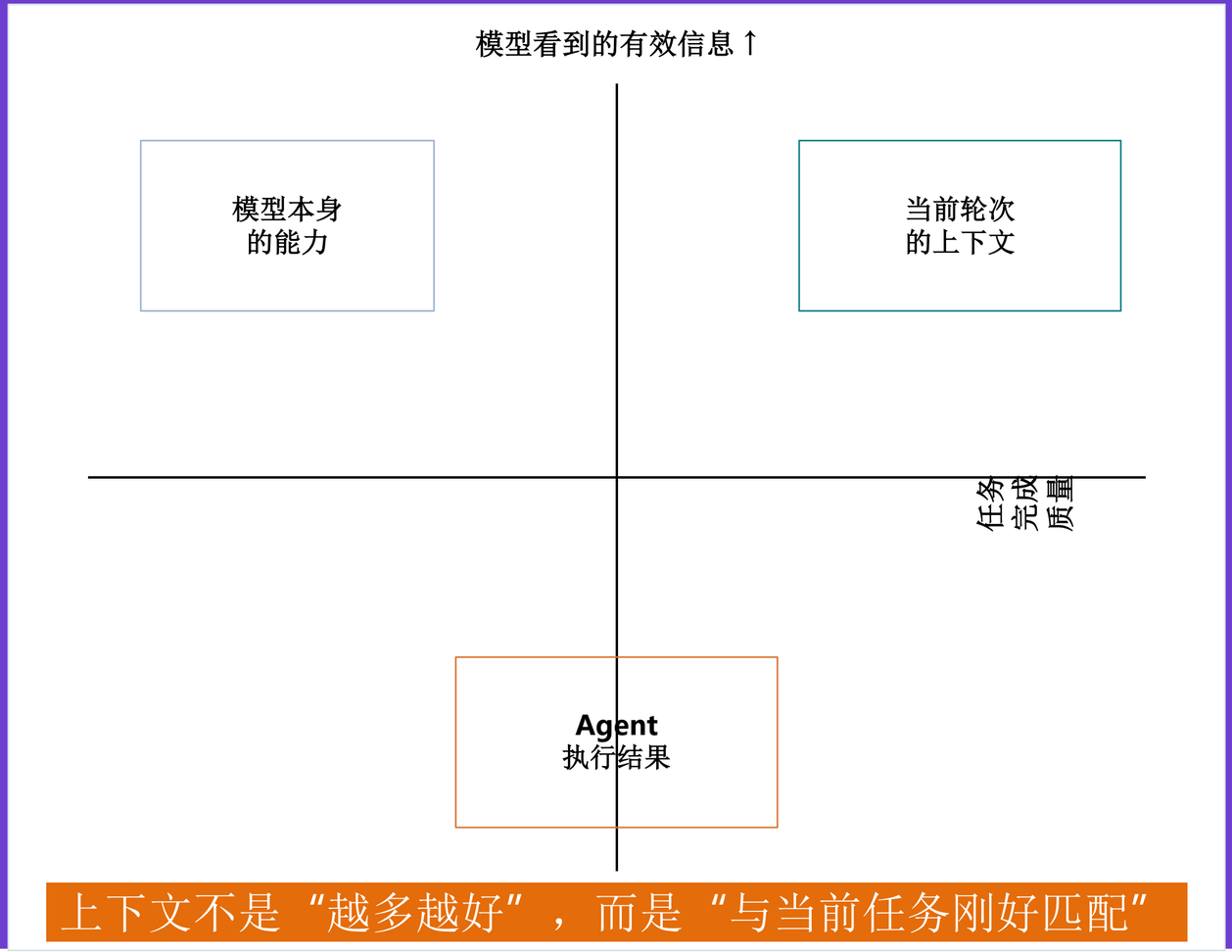
Agent 效果 = 模型能力 × 上下文质量

该给的信息要给齐

让模型知道目标、状态和边界。

不用给的信息不要给

减少噪声，保护模型注意力。



上下文不是“越多越好”，而是“与当前任务刚好匹配”

长输出先落盘，再把选择权交给模型

问题

工具输出过长

read 读取文档、bash 执行命令，可能一次撑爆上下文。

```
$ bash run_test.sh
PASS ...
...
[output continues for 18,000 lines]
```

处理

完整结果保存到本地

保留完整信息，同时避免它挤占当前轮次的注意力。

```
saved: /tmp/tool-output-042.txt
```

给模型

截断文本 + 文件路径

由模型自己判断是否需要继续读取完整内容。

```
tail: last 80 lines
file: /tmp/tool-output-042.txt
```

上下文更聚焦

模型先处理摘要和关键尾部，快速决定下一步。

完整信息仍可追溯

需要时再按路径读取，不牺牲调试能力。

压缩：提前触发，保留可继续的摘要

在模型完成上一轮任务后即判断剩余空间主动压缩；压缩结果按结构化模板保留，让模型接着做事而非重新理解。

根据剩余可用量主动判断

1. 防止超限：多轮循环让历史快速累积，需在超限前压缩。
2. 保持性能：上下文越长密度越低，模型越难抓重点。

> 实际压缩应在模型完成上一轮任务后就判断，提前为下一轮腾出空间。



↻ **低于阈值 → 主动压缩：结合模型上下文长度与当前长度，计算剩余空间。**

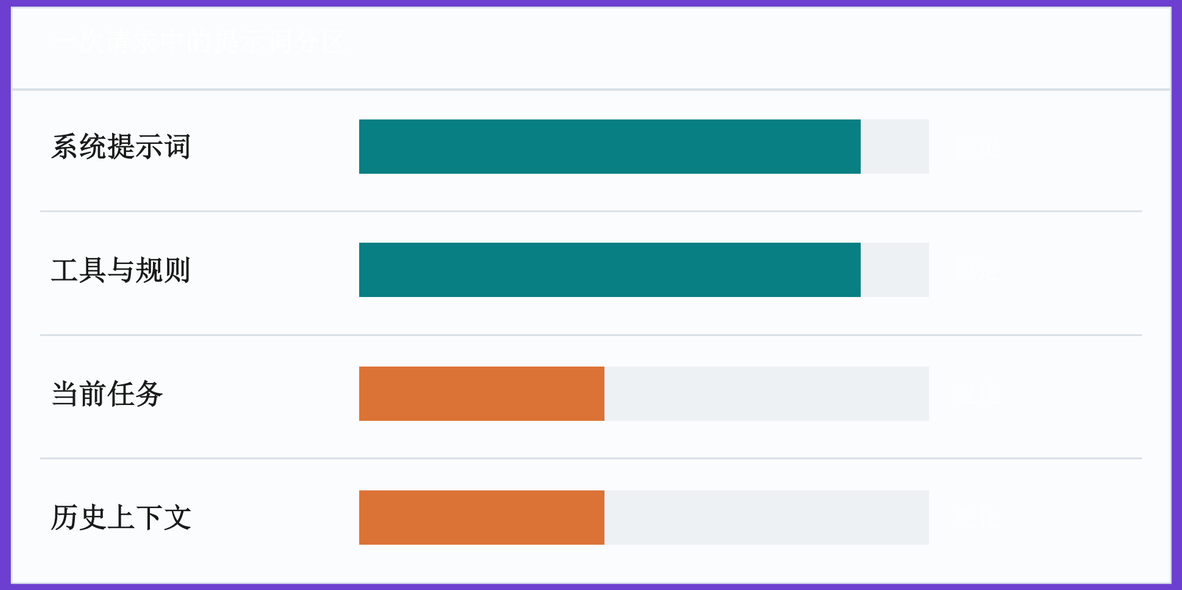
Context Summary

下一轮可直接使用

1	**用户目标**	要完成什么，成功标准是什么
2	**约束条件**	不能改变什么，必须遵守什么
3	**工作流程**	已经走到哪一步，下一步是什么
4	**关键决策**	为什么选择当前方案，哪些结论已确认
5	**下一步计划**	接下来要执行的动作与验证方式

不是删历史，而是按结构化模板提炼下一轮真正需要的信息。

稳定提示词的前缀，才能稳定命中缓存



1 合理编排分区

什么信息放前面、什么信息放后面，给稳定内容留出固定位置。

2 尽量避免前缀变化

定的系统提示词与工具定义可以反复复用，提高 prompt cache 命中率。

Context Engineering 的核心，是把有限上下文变成可复用的工作记忆。

PART 2

领域 Agent 如何做模型后训练

——破局“你先给我十万条优质数据”

01 后训练范式演变综述

范式如何演进？领域 Agent 的后训练有哪些路径？

02 Agentic RL 的核心定义、困难与解法

为什么是Agentic RL?

03 GUI测试执行Agent后训练数据集构建工作复盘

要什么样的数据，如何低成本获取

04 后训练趋势演变与相关信息摘要

总结展望：值得关注的方向与关键信息

01 后训练范式演变综述

范式如何演进？领域 Agent 的后训练有哪些路径？

后训练范式综述：奖励信号从人工走向自监督、可验证与环境交互

五种后训练范式

01 SFT
监督微调

用「输入—理想输出」示范直接教学，快速打底；只学固定套路，不优化偏好与决策。

02 RLHF
人类反馈强化学习

人工逐条比较答案、训练奖励模型，再做 RL 对齐；奖励昂贵、信号粗、只校准"语气是否像人"。

03 RLAIF
AI 反馈强化学习

把成文「宪法」（原则清单）交给模型，由模型按宪法自我批判、修订、监督，替代逐条人工标注。

04 RLVR
可验证奖励强化学习

不训练打分模型，用客观、可程序化校验的规则（断言）直接给对错当奖励。用 GRPO 在单步静态任务上激发推理。

05 Agentic RL
智能体强化学习

训练单位从单轮答案变为几十上百步的工具 / 环境交互轨迹，奖励由环境终态与多组件构成（下页详述）。

奖励信号的四级迁移

更便宜 · 更客观 · 更稠密 · 可规模化



L1 人类偏好（RLHF）
人工标注 → 训练奖励模型

L2 宪法原则（RLAIF）
模型据成文原则自批判、自监督

L3 可验证规则（RLVR）
程序断言直接判对错，无需打分模型

L4 环境反馈（Agentic RL）
终态断言 + per-turn 过程奖励，作用于整条轨迹

结论：Agentic RL 成为主战场 —— 多轮工具使用 RL 进入旗舰模型后训练主线，训练栈开源化，能力沿轨迹向小模型 / 端侧下沉

02 Agentic RL 的核心定义、困难与解法

为什么是Agentic RL?

Agentic RL 定义与关键特征

定义：Agentic RL（智能体强化学习）是在多轮轨迹上、以环境交互与终态结果为奖励信号来训练 LLM 智能体的方法。

维度	普通 RL（RLHF / RLVR）	Agentic RL
交互结构	单轮：prompt → 回答 → 打分	多轮：观察 → 动作 → 观察⋯ 多步循环，直到任务结束
环境	基本静态、无状态（题库）	动态、有状态、会被模型改变（OS、浏览器、仓库、工具 API）
动	**多轮（multi-turn）** 一条轨迹含几十到上百个 turn（思考 → 行动 → 观察），而非一次生成即结束；长程 horizon 直接决定能否学会复杂策略。	**环境交互（environment）** 行动会真实改变世界——跑代码、点网页、操作终端、读写数据库，反馈有噪声、有副作用、可复现。
卜		
信	**终态奖励（terminal reward）** "对不对"要把世界跑完才知道：fail-to-pass 测试是否转绿、文件系统/数据库/页面状态是否达标。	**工具调用（tool use）** 动作空间从生成文本进化为搜索、代码执行、GUI 键鼠操作等可组合动作；工具延迟与异步调度是系统约束。
数		
主要风险	RM 偏见、蒙对答案	环境漏洞被 reward hacking、不可复现、多轮误差累积

长 horizon 把 LLM 变成环境中的决策者，也带来两个核心难题

把 LLM 从「被动序列生成器」重新定义为嵌入动态环境、做长 horizon 决策的智能体：转移是随机的（工具返回、网页 / DB 状态），奖励稀疏且常为 terminal；与单轮、静态、可验证的 RLVR 形成对照。

01 信用分配 Credit Assignment

双重层级归因

- 哪一轮（turn）是关键?
- 该轮内又是哪些 token 的决策关键?

典型困境

任务最后失败，是第 3 步的搜索关键词错了，还是第 40 步的编辑错了？一个 terminal 信号很难回溯归因到具体某一步。

02 奖励稀疏 + 轨迹爆炸

正奖励极少

- 绝大多数路径走不到目标，拿不到正奖励

搜索空间指数膨胀

- 可选动作随步数组合增长，轨迹数量爆炸
- 信号稀疏 + 空间巨大，训练极易学不动、不收敛

小结：长 horizon 的本质矛盾 —— 一个稀疏终点分，既要回溯归因，又撑不起指数级的轨迹搜索

解法：三级奖励结构 + 可复现沙箱 + rollout harness

三类奖励结构系统比较：奖励越密、越逐 turn、且与终局一致，训练越稳、收敛越快；可复现沙箱与 rollout harness 共同把长 horizon 工程化。

01 三类奖励结构

terminal 终点一次 / delayed 阶段反馈 / per-turn 逐轮稠密；越密、越逐 turn、与终局一致，训练越稳越快。

02 rollout harness

rollout 与 trainer 解耦的异步基础设施，生成轨迹、算奖励、供策略更新（详见下表）。

03 可复现沙箱

容器 / 虚拟桌面并发，环境可重置、过程可审计。

关键基础设施 · Rollout Harness

Rollout Harness	在 RL 训练中的定位
目的	生成轨迹、算奖励、供策略更新
是否更新权重	间接：产出训练数据，不直接更新模型权重
核心组件	采样调度、环境 / 工具沙箱、多轮状态、奖励计算、异步引擎
规模	成千上万并发轨迹、长期运行
代表	veRL 等异步 RL 框架、自研 rollout 系统
主要风险	环境漏洞被 reward hacking、不可复现、训练 - 推理失配

03 GUI测试执行Agent后训练数据集构建工作复盘

要什么样的数据，如何低成本获取

蒸馏换时延：小模型本地常驻，即可承载高频测试执行

痛点：一条 GUI 用例几十上百步，每步「看截图 → 定位元素 → 生成动作」都调云端大模型，回归套件要求上千桌面并发

蒸馏前 · 现状

云端大模型 + 长 harness prompt
- 单步秒级延迟，一条用例数十次往返
- token 成本随用例步数线性增长
- 并发受 API 配额与网络约束

蒸馏后 · 目标态

本地小模型（3B–9B，边缘常驻）
- harness 规则内化进参数，prompt 大幅缩短
- 单步百毫秒级，可离线 / 私有化部署
- 支撑上千虚拟桌面并发，成本数量级下降

蒸馏对象 · 两类都要蒸

01 大模型的感知与决策能力
截图理解、元素定位、键鼠动作与断言判断

02 harness prompt 中的工程规则
定位优先级、显式等待 / 重试、弹窗处理、断言模板、失败恢复

效果证据 · 小模型追平大模型

harness annealing（harness 内化）
训练把反复调用外部 harness 的模式内化进策略，从频繁访问转为按需访问；Qwen3-8B 在 ALFWorld 达 96.9%

CUA-Gym　小模型以约 1/10 总算力，OSWorld-Verified 达 62.1% / 72.6%

WebServ　Qwen3-4B 达 55.5%，超过 Claude 4.5 Sonnet（50.0%）

规则写不尽，就用 Agentic RL 在轨迹中学会固定套路

界面状态组合爆炸、提示词越补越漏；以整条交互轨迹为训练单位，可验证奖励只是 reward 的一部分

提示词为何触顶

- 动态渲染、自定义控件、懒加载
- 随机弹窗、组件频繁改版
- 定位策略 × 异常分支组合爆炸
- SFT 只背固定示范，易过拟合特定界面



为何适合 Agentic RL

逻辑固定

登录 / 下单 / 表单 / CRUD 等流程目标明确

奖励可由多组件构成

可验证终态断言（fail-to-pass、页面 / DB 比对）是 reward 的一部分，配合 per-turn 过程奖励 / ORM

落地做法

01 **沙箱化并发环境**

自动产出可交互轨迹，CUA-Gym 3.2 万条 / 110 环境

02 **Agentic RL 轨迹优化**

GRPO / GSPO 更新整条轨迹；可验证断言 + per-turn 过程奖励组成 reward

03 **失败轨迹变新题**

WebRL 自进化课程，密集奖励防长轨迹不收敛

效果证据：WebRL 中 8B 开源模型网页任务成功率 4.8% → 42.4%，超过 GPT-4-Turbo（17.6%），且对未见过的站点可泛化

豆包工作 AI 生成

如何构建 Agentic RL 数据集

以 GUI Agent 测试执行为例

前序章节讲了"范式"与"harness"，本章回答最朴素的一问——**数据从哪来**。测试执行（GUI Agent 在真实界面里多轮点击、输入、验证）是 Agentic RL 最理想的试验田，我们用四步法把"执行过程"变成"可学习的轨迹"。

多轮 • 有状态
界面、弹窗、动态变化 = 典型 long-horizon

奖励可验证
页面状态 / 断言可程序化判定 = RLVR 友好

动作 = 工具调用
屏幕坐标 + 操作即 token + action

轨迹现场产生
非静态题库，随 rollout 生成

1

标注平台
人在回路的冷启动种子轨迹生产

2

执行日志设计
事件 + 状态快照 + 多模态证据

3

日志转训练数据
轨迹 → GRPO planning 样本

4

程序化环境合成
任务参数化 + 环境生成流水线

用事件记录执行过程，用快照保存上下文，用断言判定成败，用程序化扩规模。

轨迹即数据，训练场即考场

Agentic RL 把训练单位从"一道题的单轮回答"变成"一条几十上百步的交互轨迹"。数据不再是离线字符串比对，而是**训练时现场 rollout 产生**的、由环境终态判分的交互记录。

维度	普通 RL（RLHF / RLVR）	Agentic RL
交互结构	单轮：prompt→回答→打分	多轮：观察→动作→观察… 直到结束
环境	静态、无状态题库	动态、有状态，被模型改变
动作空间	输出 token	token + 工具调用 / 环境动作
奖励	结果对错 / 人类偏好，多在终点	终态目标 + 每步环境反馈
数据来源	静态数据集	训练时现场 rollout 产生

长 horizon 决定上限

一条轨迹含几十到上百个 turn（思考→行动→观察），长 horizon 决定能否学会复杂策略。

可验证奖励是关键

"对不对"要把世界跑完才知道：fail-to-pass 测试转绿、页面 / DB 状态达标。

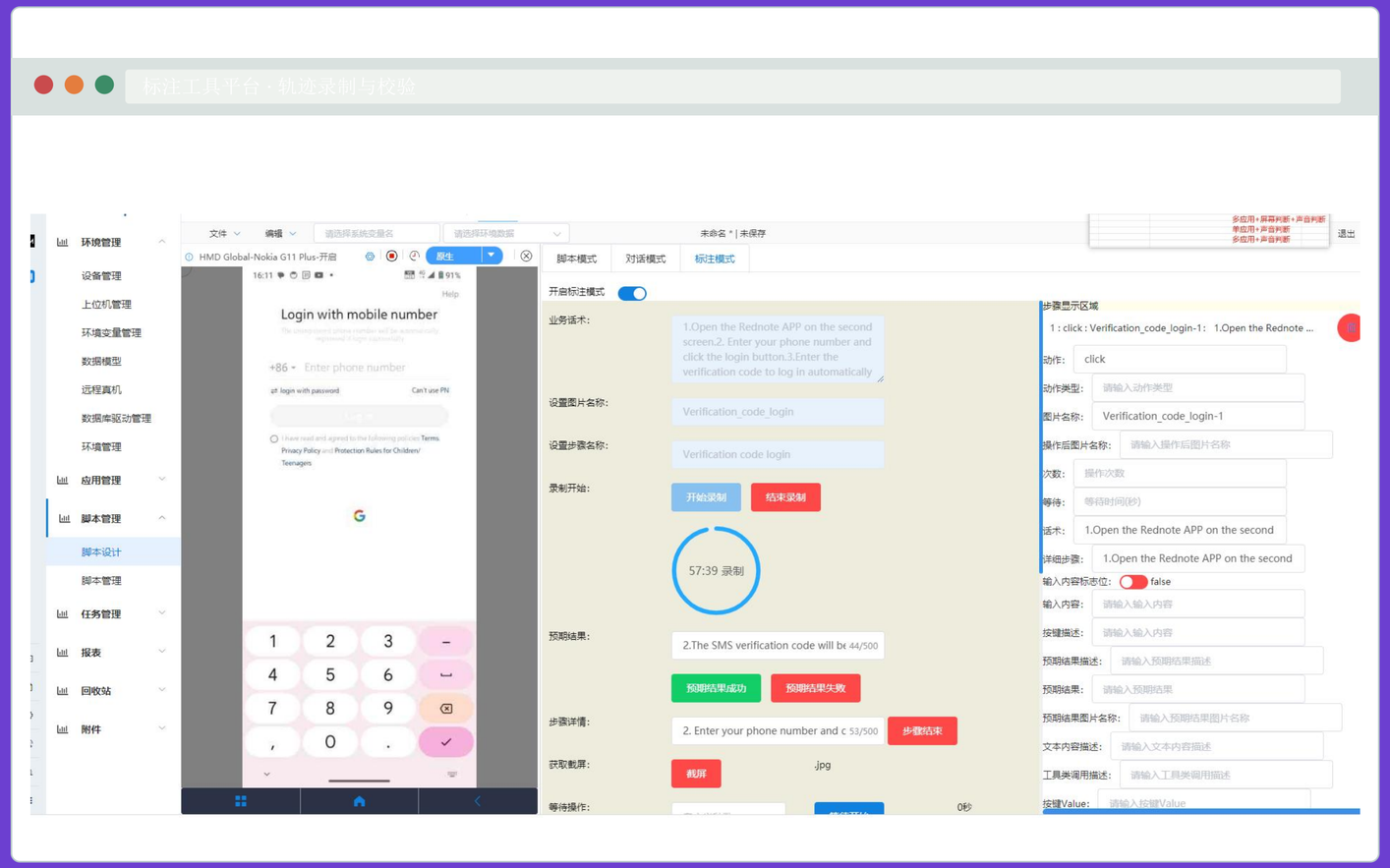
训练即评测

同一份镜像与断言，既当 rollout 奖励、也当评测分——训练与评测共用一套验收。

数据飞轮：标注 / 人工轨迹（SFT 冷启动）→ 在线 RL rollout 轨迹 → 经验沉淀（tips / shortcuts）→ 回灌课程。GUI Agent 每轮"感知-规划-动作-反思"即是天然轨迹样本。

人在回路的数据起点

标注平台是整条飞轮的"种子"——贵但准。它产出高质量成功轨迹用于 SFT 冷启动，并把专家操作习惯显式化为 tips / shortcut。



冷启动轨迹： 人工 / 半自动示范"怎么完成一个测试任务"

多模态对齐： 截图 + OCR + 坐标 + 操作意图一一对应

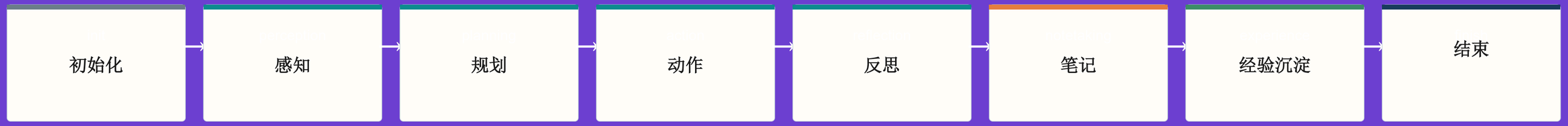
质量闸门： 人工校验轨迹、纠正感知 / 规划错误

经验显式化： 专家习惯沉淀为可复用 shortcut

instruction 自然语言任务指令	**screenshot** 每步屏幕证据
action_object 结构化动作（Tap/Type...）	**action_thought** 操作意图与理由
subgoal 当前子目标	**outcome** 动作反思结果

不是文本日志，是事件 + 状态 + 证据

日志面向智能体执行过程，采用**事件溯源 + 状态快照 + 多模态证据**三位一体，支撑可追溯、可调试、可重放。



中段 **感知 → 规划 → 动作 → 反思** 构成闭环，反思结果反馈给下一轮规划；max_itr / max_consecutive_failures 防止无限循环

事件描述"做了什么"（action_object），状态快照描述"当时知道什么"（info_pool：历史、计划、子目标、错误、进度）。既能分析轨迹，也能还原任一时刻上下文。

可追溯

① 当前轮上下文（summary / action_history）② 任务状态（important_notes / progress）③ 跨任务经验（tips.txt / shortcuts.json，如 Tap→Type→Enter 封装为快捷动作）。

跨任务学习

prompt_* · raw_response · 结构化字段并存，可定位错误究竟发生在感知 / 规划 / 格式 / 解析 / 执行 / 判定哪一环；动作结构化保存支持离线重放。

可调试 可重放

TRAJECTORY → GRPO SAMPLE

从执行轨迹到 GRPO 样本

脚本把 MobileAgent 的成功执行日志转换为 GRPO planning 数据集：**只采成功轨迹**作正样本，按"最近截图 + 规划 prompt + 模型原始回复 + 子目标 / 思维链 / 计划"结构抽取。

① 收集成功日志
uccess_file 读路径，路径前缀重映射到本机挂载

→

② 解析 **steps.json**
单任务 / 批量子任务两种目录结构

→

③ 提取 **planning** 落盘
为可被 GRPO 框架读取的 JSON

字段	来源	用途
picture_file	最近 perception 截图	多模态图像输入
action_prompt	prompt_planning	模型输入 X
response	raw_response	教师信号 Y
current_subgoal	current_subgoal	当前子目标
planning_thought	thought	思维链 CoT
planning_description	plan	规划描述

```
                    # 状态延续：planning 借最近一次 perception 的截图
for step in steps_list:
    if step.get('operation') == 'perception':
        current_pic = step['screenshot']
    if step.get('operation') == 'planning':
        res_list.append({
            'picture_file': current_pic,
            'action_prompt': step['prompt_planning'],
            'response': step['raw_response'],
            ...
        })
```

关键设计：聚焦 planning 而非 action（注释掉了 action 分支）→ 训练目标是子目标规划能力。(picture_file + action_prompt, response) 构成标准 SFT 式 (X, Y) 对，可作 GRPO 的 base / reference 初始化。

让任务"造"出来，而非"标"出来

人工标注贵、静态基准易被背、RL 需要成千上万可执行任务——环境本身必须流水线化。三条路线把"题库"变成"题厂"。

标注平台播种 → 日志标准化 → 样本化 → 程序化扩规模

任务模板参数化

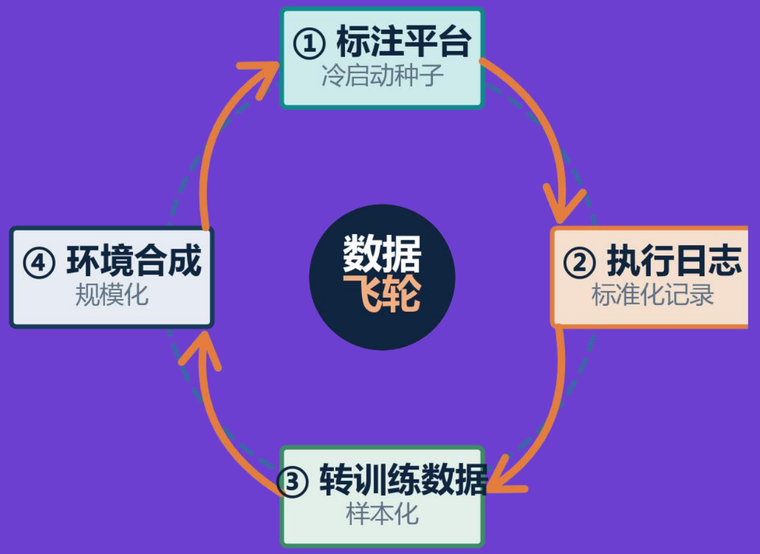
个模板换实例说法无限生成；每任务配 init / success / teardown 脚本，保证可复现、可自动判分。同一模板可无限换实例，对抗"背答案"。

环境生成流水线

从真实 PR / issue 反向"反译"成任务，LLM agent 自动克隆仓库、推断依赖、生成可复现 Docker 镜像并把测试跑通，把"手工配环境"自动化。

自进化课程

把智能体失败尝试自动改写成新任务，源源不断造题；结果奖励模型在稀疏反馈

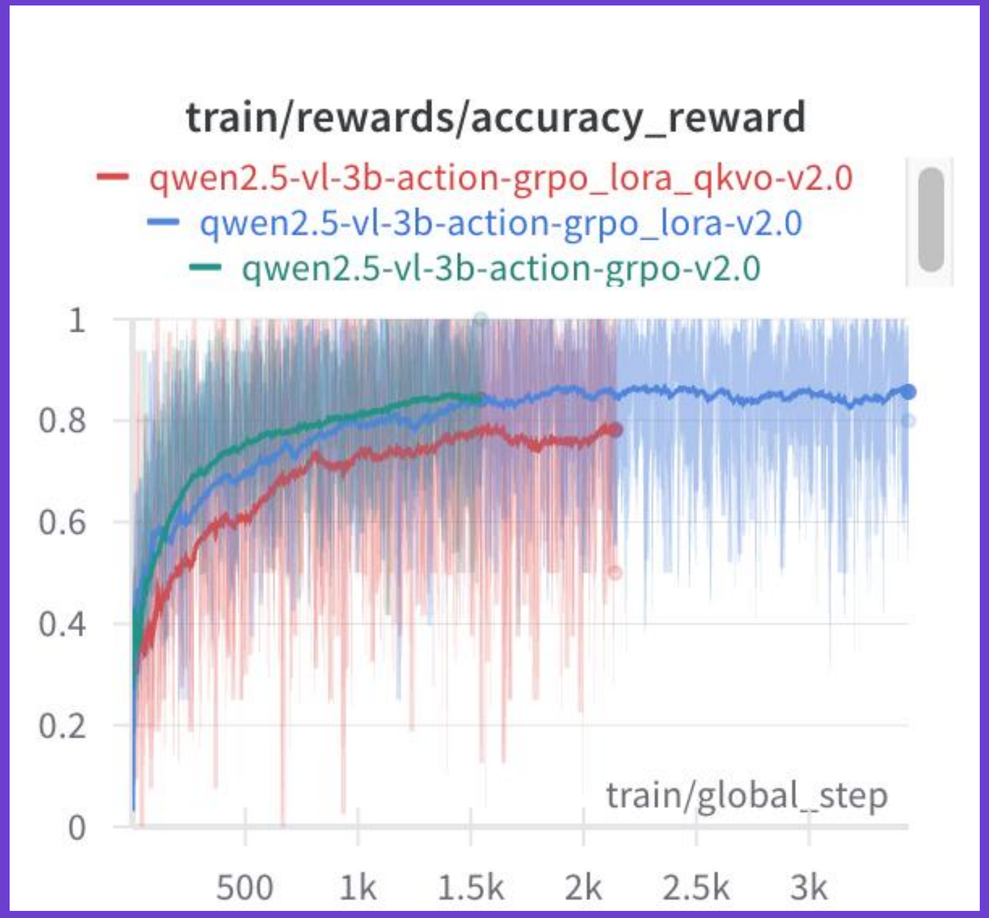


SEED → STANDARDIZE → SAMPLE → SCALE

测试执行映射：测试用例模板化 → 参数化生成大量"点击 → 输入 → 断言"任务；init/success/teardown 保证可复现 + 可判分；每实例带生产日期标签，只拿晚于模型训练截止日的题做干净评测。

考核题（评测）与练习题（训练）共用同一套验收标准 —— 训练场即考场。

部分实验结果



有特殊提示词+正确动作和截图 vs 无特殊提示词+动作训练

历史正确记录 vs 训练后是否还正确

基座模型	训练方式	freeze_vision	训练显卡	显存占用	训练集acc	验证集acc
Qwen2.5-VL-3B-Instruct	未训练					0.54
Qwen2.5-VL-3B-Instruct	全参	false	A6000	3卡：150G	0.85	0.82

模型	训练集准确率	验证集准确率
Qwen2.5–1.5B–Instruct	17.31%	15.19%
DeepSeek–R1–Distill–Qwen–1.5B	44.23%	40.51%
Qwen2.5–1.5B–Instruct–GPRO–1500steps	94.23%	88.61%

Qwen2.5-VL

Qwen2.5-VL

- 1500steps训练后，准确率提升48%左右

在 planning 数据上做 GRPO 训练，模型子目标规划能力随步数稳定提升，任务成功率随之上升——验证"轨迹即数据"路线有效。

四句话带走

把 GUI Agent 测试执行变成 Agentic RL 数据集，本质是把"执行过程"变成"可学习的轨迹"。

01 **测试执行天然契合 Agentic RL**

多轮、有状态、奖励可验证、轨迹现场产生——四特征同时满足，是 long-horizon RL 的理想试验田。

02 **数据观：轨迹即数据，训练场即考场**

数据来自 rollout 而非静态集；同一份镜像与断言，既是奖励信号也是评测分数。

03 **日志设计是地基**

事件 + 状态快照 + 多模态证据三位一体，支撑可追溯、可调试、可重放与跨任务经验沉淀。

04 **规模化靠程序化**

任务参数化 + 环境生成流水线 + 自进化课程，配生产日期标签防污染，把"题库"变成"题厂"。

04 后训练趋势演变与相关信息摘要

总结展望：值得关注的方向与关键信息

后训练正在从"对齐回答"转向"训练可执行的智能体"

核心变化：奖励从人类偏好迁移到程序验证与环境反馈，训练单位从单轮答案升级为多轮交互轨迹。

奖励信号四级迁移

人类偏好 → AI judge → 可验证奖励 → 环境反馈。数学/代码依赖单测与证明检查，GUI/agent 任务依赖终端或沙箱终态。

Agentic RL 成为主战场

长 horizon、随机环境转移和工具调用带来两项核心工程问题：turn 级信用分配与异步 rollout。

01 轨迹化训练与 turn 级奖励

训练从单轮答案变成多轮轨迹。dense、逐 turn 且与终局一致的奖励通常更稳。

SWEET-RL / iStar / TRACE

02 全异步训练栈成默认基础设施

rollout 与 trainer 拆池、队列解耦、容忍陈旧数据，解决工具等待和长尾轨迹造成的 GPU 空转。

AReaL、veRL、ROLL、Qwen3.5

03 动态环境与任务生成军备竞赛

新题持续收割、环境自动生成、终态自动验收组合起来，用于防污染与规模化 rollout。

SWE-bench-Live、WebGym、SIMA 2

04 GUI / Computer Use 新边疆

视觉观察 + 鼠标键盘动作进入 RL，GUI 逐步与命令行共享统一动作空间。

OpenAI CUA、UI-TARS-2、Qwen-UI

05 自进化、自博弈与世界模型内生化

模型同时生成任务、经验或过程监督，形成更强的自举闭环。

SIMA 2、ECHO、MiMo-V2.6

06 RL 算力向预训练量级靠拢

Live RL、长时训练和大规模 rollout 正重写后训练的成本结构。

Grok 4、混元-T1、MiMo-V2.6

07 "小模型 + Agent"下沉端侧

能力蒸馏、工具调用和端侧推理结合，降低实时 agent 的延迟与部署成本。

Nemotron、MiniCPM5-2B、Step 3.5

环境沙箱列表 - 按领域

环境同时承担"训练场"和"考场"：每个领域列出 1–2 个典型环境，突出其任务类型与验证方式。

领域	代表环境	任务规模	部署方式	判分方式	定位
代码工程	WE-rebench V2	32,079 · 20 语言	预置 Docker 镜像	测试断言 + 日期元数据	**主流训练环境**
	WE-bench Verified	500 题	仓库快照 / Docker	fail-to-pass 单测	**严格评测锚点**
终端	Terminal-Bench 2.0	89 题	隔离 Docker	人工 oracle + 容器测试	评测与训练并用
数据库 / 工具	τ-bench	165 题	进程内 Python 模拟	终态 DB 逐字段比对	**可直接 rollout 训练**
浏览器	WebArena	812 题	自托管 Docker 站点	网站 DB / 页面断言	可训练、可评测
	WebGym	10 万级	容器并发隔离	可验证终态	**大规模训练环境**
桌面 / 移动 / 综合	OSWorld	369 题	Ubuntu / Windows / macOS VM	程序化成功函数	**GUI rollout 训练**
	AndroidWorld	116 题（参数化）	Android 模拟器	系统状态检查脚本	**移动端训练环境**

环境设计原则 训练与评测保持"同源不同题"；奖励优先采用测试转绿、数据库字段一致、文件系统状态匹配等终态可执行断言；把日期元数据、镜像哈希和轨迹核验纳入环境本身，降低数据污染与 reward hacking。

THANKS

演讲嘉宾：詹伶俐